5.3 Using the Tracing System

The whole point of simulation is to generate output for further study, and the ns-3 tracing system is a primary mechanism for this. Since ns-3 is a C++ program, standard facilities for generating output from C++ programs could be used:

```
#include <iostream>
...
int main ()
{
  ...
  std::cout << "The value of x is " << x << std::endl;
  ...
}
```

You could even use the logging module to add a little structure to your solution. There are many well-known problems generated by such approaches and so we have provided a generic event tracing subsystem to address the issues we thought were important.

The basic goals of the ns-3 tracing system are:

- For basic tasks, the tracing system should allow the user to generate standard tracing for popular tracing sources, and to customize which objects generate the tracing;
- Intermediate users must be able to extend the tracing system to modify the output format generated, or to insert new tracing sources, without modifying the core of the simulator;
- Advanced users can modify the simulator core to add new tracing sources and sinks.

The ns-3 tracing system is built on the concepts of independent tracing sources and tracing sinks, and a uniform mechanism for connecting sources to sinks. Trace sources are entities that can signal events that happen in a simulation and provide access to interesting underlying data. For example, a trace source could indicate when a packet is received by a net device and provide access to the packet contents for interested trace sinks.

Trace sources are not useful by themselves, they must be “connected” to other pieces of code that actually do something useful with the information provided by the sink. Trace sinks are consumers of the events and data provided by the trace sources. For example, one could create a trace sink that would (when connected to the trace source of the previous example) print out interesting parts of the received packet.

The rationale for this explicit division is to allow users to attach new types of sinks to existing tracing sources, without requiring editing and recompilation of the core of the simulator. Thus, in the example above, a user could define a new tracing sink in her script and attach it to an existing tracing source defined in the simulation core by editing only the user script.

In this tutorial, we will walk through some pre-defined sources and sinks and show how they may be customized with little user effort. See the ns-3 manual or how-to sections for information on advanced tracing configuration including extending the tracing namespace and creating new tracing sources.

5.3.1 ASCII Tracing

Ns-3 provides helper functionality that wraps the low-level tracing system to help you with the details involved in configuring some easily understood packet traces. If you enable this functionality,

you will see output in a ASCII files — thus the name. For those familiar with ns-2 output, this type of trace is analogous to the out.tr generated by many scripts.

Let's just jump right in and add some ASCII tracing output to our scratch/myfirst.cc script. Right before the call to Simulator::Run (), add the following lines of code:

```
AsciiTraceHelper ascii;
pointToPoint.EnableAsciiAll (ascii.CreateFileStream ("myfirst.tr"));
```

Like in many other ns-3 idioms, this code uses a helper object to help create ASCII traces. The second line contains two nested method calls. The "inside" method, CreateFileStream() uses an unnamed object idiom to create a file stream object on the stack (without an object name) and pass it down to the called method. We'll go into this more in the future, but all you have to know at this point is that you are creating an object representing a file named "myfirst.tr" and passing it into ns-3. You are telling ns-3 to deal with the lifetime issues of the created object and also to deal with problems caused by a little-known (intentional) limitation of C++ ofstream objects relating to copy constructors.

The outside call, to EnableAsciiAll(), tells the helper that you want to enable ASCII tracing on all point-to-point devices in your simulation; and you want the (provided) trace sinks to write out information about packet movement in ASCII format.

For those familiar with ns-2, the traced events are equivalent to the popular trace points that log "+", "-", "d", and "r" events.

You can now build the script and run it from the command line:

```
./waf --run scratch/myfirst
```

Just as you have seen many times before, you will see some messages from Waf and then "'build' finished successfully" with some number of messages from the running program.

When it ran, the program will have created a file named myfirst.tr. Because of the way that Waf works, the file is not created in the local directory, it is created at the top-level directory of the repository by default. If you want to control where the traces are saved you can use the --cwd option of Waf to specify this. We have not done so, thus we need to change into the top level directory of our repo and take a look at the ASCII trace file myfirst.tr in your favorite editor.

5.3.1.1 Parsing Ascii Traces

There's a lot of information there in a pretty dense form, but the first thing to notice is that there are a number of distinct lines in this file. It may be difficult to see this clearly unless you widen your window considerably.

Each line in the file corresponds to a *trace event*. In this case we are tracing events on the *transmit queue* present in every point-to-point net device in the simulation. The transmit queue is a queue through which every packet destined for a point-to-point channel must pass. Note that each line in the trace file begins with a lone character (has a space after it). This character will have the following meaning:

- +: An enqueue operation occurred on the device queue;

- -: A dequeue operation occurred on the device queue;
- d: A packet was dropped, typically because the queue was full;
- r: A packet was received by the net device.

Let's take a more detailed view of the first line in the trace file. I'll break it down into sections (indented for clarity) with a two digit reference number on the left side:

```
00 +
01 2
02 /NodeList/0/DeviceList/0/$ns3::PointToPointNetDevice/TxQueue/Enqueue
03 ns3::PppHeader (
04   Point-to-Point Protocol: IP (0x0021))
05   ns3::Ipv4Header (
06     tos 0x0 ttl 64 id 0 protocol 17 offset 0 flags [none]
07     length: 1052 10.1.1.1 > 10.1.1.2)
08     ns3::UdpHeader (
09       length: 1032 49153 > 9)
10       Payload (size=1024)
```

The first line of this expanded trace event (reference number 00) is the operation. We have a + character, so this corresponds to an *enqueue* operation on the transmit queue. The second line (reference 01) is the simulation time expressed in seconds. You may recall that we asked the UdpEchoClientApplication to start sending packets at two seconds. Here we see confirmation that this is, indeed, happening.

The next line of the example trace (reference 02) tell us which trace source originated this event (expressed in the tracing namespace). You can think of the tracing namespace somewhat like you would a filesystem namespace. The root of the namespace is the NodeList. This corresponds to a container managed in the ns-3 core code that contains all of the nodes that are created in a script. Just as a filesystem may have directories under the root, we may have node numbers in the NodeList. The string /NodeList/0 therefore refers to the zeroth node in the NodeList which we typically think of as "node 0". In each node there is a list of devices that have been installed. This list appears next in the namespace. You can see that this trace event comes from DeviceList/0 which is the zeroth device installed in the node.

The next string, $ns3::PointToPointNetDevice tells you what kind of device is in the zeroth position of the device list for node zero. Recall that the operation + found at reference 00 meant that an enqueue operation happened on the transmit queue of the device. This is reflected in the final segments of the "trace path" which are TxQueue/Enqueue.

The remaining lines in the trace should be fairly intuitive. References 03-04 indicate that the packet is encapsulated in the point-to-point protocol. References 05-07 show that the packet has an IP version four header and has originated from IP address 10.1.1.1 and is destined for 10.1.1.2. References 08-09 show that this packet has a UDP header and, finally, reference 10 shows that the payload is the expected 1024 bytes.

The next line in the trace file shows the same packet being dequeued from the transmit queue on the same node.

The Third line in the trace file shows the packet being received by the net device on the node with the echo server. I have reproduced that event below.

```
00 r
01 2.25732
02 /NodeList/1/DeviceList/0/$ns3::PointToPointNetDevice/MacRx
03   ns3::Ipv4Header (
04     tos 0x0 ttl 64 id 0 protocol 17 offset 0 flags [none]
05     length: 1052 10.1.1.1 > 10.1.1.2)
06     ns3::UdpHeader (
07       length: 1032 49153 > 9)
08       Payload (size=1024)
```

Notice that the trace operation is now `r` and the simulation time has increased to 2.25732 seconds. If you have been following the tutorial steps closely this means that you have left the `DataRate` of the net devices and the channel `Delay` set to their default values. This time should be familiar as you have seen it before in a previous section.

The trace source namespace entry (reference 02) has changed to reflect that this event is coming from node 1 (`/NodeList/1`) and the packet reception trace source (`/MacRx`). It should be quite easy for you to follow the progress of the packet through the topology by looking at the rest of the traces in the file.

5.3.2 PCAP Tracing

The `ns-3` device helpers can also be used to create trace files in the `.pcap` format. The acronym pcap (usually written in lower case) stands for *p*acket *cap*ture, and is actually an API that includes the definition of a `.pcap` file format. The most popular program that can read and display this format is Wireshark (formerly called Ethereal). However, there are many traffic trace analyzers that use this packet format. We encourage users to exploit the many tools available for analyzing pcap traces. In this tutorial, we concentrate on viewing pcap traces with tcpdump.

The code used to enable pcap tracing is a one-liner.

```
pointToPoint.EnablePcapAll ("myfirst");
```

Go ahead and insert this line of code after the ASCII tracing code we just added to `scratch/myfirst.cc`. Notice that we only passed the string “myfirst,” and not “myfirst.pcap” or something similar. This is because the parameter is a prefix, not a complete file name. The helper will actually create a trace file for every point-to-point device in the simulation. The file names will be built using the prefix, the node number, the device number and a “.pcap” suffix.

In our example script, we will eventually see files named “myfirst-0-0.pcap” and “myfirst-1-0.pcap” which are the pcap traces for node 0-device 0 and node 1-device 0, respectively.

Once you have added the line of code to enable pcap tracing, you can run the script in the usual way:

```
./waf --run scratch/myfirst
```

If you look at the top level directory of your distribution, you should now see three log files: `myfirst.tr` is the ASCII trace file we have previously examined. `myfirst-0-0.pcap` and `myfirst-1-0.pcap` are the new pcap files we just generated.

5.3.2.1 Reading output with tcpdump

The easiest thing to do at this point will be to use `tcpdump` to look at the `pcap` files.

```
tcpdump -nn -tt -r myfirst-0-0.pcap
reading from file myfirst-0-0.pcap, link-type PPP (PPP)
2.000000 IP 10.1.1.1.49153 > 10.1.1.2.9: UDP, length 1024
2.514648 IP 10.1.1.2.9 > 10.1.1.1.49153: UDP, length 1024

tcpdump -nn -tt -r myfirst-1-0.pcap
reading from file myfirst-1-0.pcap, link-type PPP (PPP)
2.257324 IP 10.1.1.1.49153 > 10.1.1.2.9: UDP, length 1024
2.257324 IP 10.1.1.2.9 > 10.1.1.1.49153: UDP, length 1024
```

You can see in the dump of `myfirst-0-0.pcap` (the client device) that the echo packet is sent at 2 seconds into the simulation. If you look at the second dump (`myfirst-1-0.pcap`) you can see that packet being received at 2.257324 seconds. You see the packet being echoed back at 2.257324 seconds in the second dump, and finally, you see the packet being received back at the client in the first dump at 2.514648 seconds.

5.3.2.2 Reading output with Wireshark

If you are unfamiliar with Wireshark, there is a web site available from which you can download programs and documentation: http://www.wireshark.org/.

Wireshark is a graphical user interface which can be used for displaying these trace files. If you have Wireshark available, you can open each of the trace files and display the contents as if you had captured the packets using a *packet sniffer*.